Polymath15, second thread: generalising the Riemann-Siegel approximate functional equation

Terence Tao · 2 February 2018

https://terrytao.wordpress.com/2018/02/02/polymath15-second-thread-generalising-the-riemann-siegel-approximate-functional-equation/

tao2typ · converted to Typst 2026-10-14 (UTC)

This is the second "research" thread of the Polymath15 project to upper bound the de Bruijn-Newman constant Λ, continuing this previous thread. Discussion of the project of a non-research nature can continue for now in the existing proposal thread. Progress will be summarised at this Polymath wiki page.

We now have the following proposition (see this page for a proof sketch) that looks like it can give a numerically feasible approach to bound Λ:

> **Proposition 1** Suppose that one has parameters $t_0, T, \varepsilon > 0$ obeying the following properties:
>
> - All the zeroes of $H_0(x+iy) = 0$ with $0 \leq x \leq T$ are real.
> - There are no zeroes $H_t(x+iy) = 0$ with $0 \leq t \leq t_0$ in the region $\{x+iy : x \geq T; 1 - 2t \geq y^2 \geq \varepsilon^2 + (T-x)^2\}$.
> - There are no zeroes $H_{t_0}(x+iy) = 0$ with $x > T$ and $y \geq \varepsilon$.
>
> Then one has $\Lambda \leq t_0 + \frac{1}{2}\varepsilon^2$.

The first hypothesis is already known for T up to about 10^{12} (we should find out exactly what we can reach here). Preliminary calculations suggest that we can obtain the third item provided that $t_0, \varepsilon \gg \frac{1}{\log T}$. The second hypothesis requires good numerical calculation for H_t, to which we now turn.

The initial definition of H_t is given by the formula

$$H_t(z) := \int_0^\infty e^{tu^2} \Phi(u) \cos(zu)\ du$$

where

$$\Phi(u) := \sum_{n=1}^\infty (2\pi^2 n^4 e^{9u} - 3\pi n^2 e^{5u}) \exp(-\pi n^2 e^{4u}).$$

This formula has proven numerically computable to acceptable error up until about the first hundred zeroes of H_t, but degrades after that, and so other exact or approximate formulae for H_t are needed. One possible exact formula that could be useful is

$$H_t(z) = \frac{1}{2}(K_{t,\theta}(z) + \overline{K_{t,\theta}(\overline{z})})$$

where

$$K_{t,\theta}(z) := \sum_{n=1}^\infty (2\pi^2 n^4 I_{t,\theta}(z - 9i, \pi n^2) - 3\pi n^2 I_{t,\theta}(z - 5i, \pi n^2))$$

and

$$I_{t,\theta}(b,\beta) := \int_{i\theta}^{i\theta+i\infty} \exp(tu^2 - \beta e^{4u} + ibu)\ du$$

and $-\pi/8 < \theta < \pi/8$ can be chosen arbitrarily. We are still trying to see if this can be implemented numerically to give better accuracy than the previous formula.

It seems particularly promising to develop a generalisation of the Riemann-Siegel approximate functional equation for H_0. Preliminary computations suggest in particular that we have the approximation

$$H_t(x+iy) \approx \frac{1}{4}(F_t(\frac{1+ix-y}{2}) + \overline{F_t(\frac{1+ix+y}{2})})$$

where

$$F_t(s) := \pi^{-s/2}\Gamma(\frac{s+4}{2}) \sum_{n \leq \sqrt{\mathrm{Im}(s)/2\pi}} \frac{\exp(\frac{t}{16}\log^2 \frac{s+4}{2\pi n^2})}{n^s}.$$

Some very preliminary numerics suggest that this formula is reasonably accurate even for moderate values of x, though further numerical verification is needed. As a proof of concept, one could take this approximation as exact for the purposes of seeing what ranges of T one can feasibly compute with (and for extremely large values of T, we will presumably have to introduce some version of the Odlyzko-Schönhage algorithm. Of course, to obtain a rigorous result, we will eventually need a rigorous version of this formula with explicit error bounds. It may also be necessary to add more terms to the approximation to reduce the size of the error.

Sujit Nair has kindly summarised for me the current state of affairs with the numerics as follows:

—

- We need a real milestone and work backward to set up intermediate goals. This will definitely help bring in focus!
- So far, we have some utilities to compute zeroes of H_t with a nonlinear solver which uses roots of H_0 as an initial condition. The solver is a wrapper around MINPACK's implementation of Powell's method. There is some room for optimization. For example, we aren't providing the solver with the analytical Jacobian which speeds up the computation and increases accuracy.
- We have some results in the output folder which contains the first 1000 roots of H_t for some small values of $t \in \{0.01, 0.1, 0.22\}$, etc. They need some more organization and visualization.

We have a decent initial start but we have some ways to go. Moving forward, here is my proposition for some areas of focus. We should expand and prioritize after some open discussion.

1. Short term Optimize the existing framework and target to have the first million zeros of H_t (for a reasonable range of t) and the corresponding plots. With better engineering practice and discipline, I am confident we can get to a few tens of millions range. Some things which will help include parallelization, iterative approaches (using zeroes of H_t to compute zeroes of $H_{t+\delta t}$), etc.
2. Medium term We need to explore better ways to represent the zeros and compute them. An analogy is the computation of Riemann zeroes up to height T. It is computed by computing the sign changes of $Z(t)$ (page 119 of Edwards) and by exploiting the $\sqrt{T}$ speed up with the Riemann-Siegel formulation (over Euler-Maclaurin). For larger values of j, I am not sure the root solver based approach is going to work to understand the gaps between zeroes.
3. Long term We also need a better understanding of the errors involved in the computation — truncation, hardware/software, etc.

Comments (102)

 **Terence Tao** · 2 February 2018

One medium-term numerical goal which would be helpful would be to gather some data regarding the accuracy of the Riemann-Siegel type approximations. When $t = 0$, the error term in these approximations is about $O(x^{-1/2})$ times the main term; hopefully a similar accuracy is present for $t > 0$ as well. If we are lucky, that is enough accuracy that it will be justifiable (for a preliminary pass at least) to pretend that the Riemann-Siegel expansion is in fact exact, and then try to control zeroes of that expansion at large height.

 **KM** · 3 February 2018

I ran some quick tests using the original Riemann Siegel eqns (as implemented in Gourdon's work), and checked the effect of using the first few error terms. As expected, the difference is significant and even the estimates of the smaller zeroes become much better. It would be great to have corresponding first few error terms for Ht.

Also, there seem to be practical considerations with the root finding algorithms using initial guesses. If there are many close roots in an interval, then it's difficult to trivially control which root will the algorithm converge to. Which results in occasionally the algorithm converging to the same root from multiple nearby guesses, and missing a few in between. It can be dealt with by carefully and iteratively expanding the interval where a root should be searched for, till the algorithm finds a sign change.

But then it's unclear whether this approach is less or more expensive than the original approach of simply evaluating H_t at close enough regularly spaced intervals and then counting the sign changes.

Will try out the counting approach between two large T heights for some t values.

 **KM** · 3 February 2018

Oh, just noticed you have mentioned the root solving limitations in the last section.

 **meditationatae** · 3 February 2018

Perhaps Gauss-Legendre quadrature, a method in the family of Gaussian quadrature methods, could be useful in obtaining a few high-accuracy values of H_t(z), for example to validate other computation methods. A reference at Wikipedia: https://en.wikipedia.org/wiki/Gaussian_quadrature .

I experimented with trying to compute H_0 (2*z_100), where z_100 is the imaginary part of the 100th zeta zero, using the first exact formula. Using 5120 intervals of equal lengths, a length of 1/1600 as it turns out, and a 100-point Gauss-Legendre method on each interval, the command in PARI/gp returned a numerical value of roughly 1 E-156 after 31 minutes. For comparison, H_0 (2*z_100 + 1) returns roughly 9.9 E-60, much larger than at 2*z_100; this is to be expected if the integration of the very oscillatory H_0 function is highly accurate, since 2*z_100 is a zero of H_0 .

I should add that I've only used Gaussian quadrature methods on a couple of occasions, and that it's new to me...

 **meditationatae** · 3 February 2018

I find that the Gauss-Legendre method using a 400-point rule and 32 intervals of length 1/16 delivers the same accuracy as above in computing H_0(2*z_100), but in only 15 seconds. This includes the time spent initializing the tables of weights and abscissas for the 400-point Gauss-Legendre rule.

For illustration and reference, the PARI/gp command entered is copied below:

```
intnumgaussinit(400);
 s = 0.0; x=2*z100 ;
 for(J=0,32, s=s+intnumgauss(X=J/16,(J+1)/16, Phi(X)*cos(x*X),));
 s
```

Anonymous · 3 February 2018

A possible approach for such approximation to $H_t(z)$ is to use its Taylor series in powers of t
$H_t(z) = \sum_{n\geq 0} a_n(z)t^n$
Where $a_n(z) := (1/n!)\int_0^\infty u^{2n}\Phi(u)\cos(zu)du$
Since $a_0(z) = H_0(z)$, and $H_t(z)$ is entire (in both z and t), the Taylor coefficients $a_n(z)$ (which are entire functions) are super exponentially decaying, so in order to estimate $H_t(z)$ by truncating its Taylor series, the required number of terms should depend on the required precision δ(say), z and t and is expected to be roughly proportional to $\log(1/\delta)/\log(1/t)$.
It seems that the asymptotic approximations of $a_n(z)$ (via their defining integrals) should be similar to that of $a_0(z)$.

Anonymous · 3 February 2018

From the integral representation of $a_n(z)$ (or directly from the backwards heat equation for $H_t(z)$), it follows that
$a_n(z) = (-1/n)a''_{n-1}(z)$, $n \geq 1$
So it seems that the asymptotic expansions of the coefficients $a_n(z)$ can be computed recursively from that of $a_0(z) = H_0(z)$.

Anonymous · 3 February 2018

This approach can be used for each given disk $|z - z_0| \leq r$ with an explicit analytic approximation $F(z)$ of $H_0(z)$ and with explicit (uniform) upper bound $M(E)$ on the magnitude of the approximation error $E = H_0 - F$ on the given disk, to derive for each (complex) t an explicit analytic approximation $F_t(z)$ to $H_t(z)$ with an explicit (uniform) bound $M(E_t)$ on the magnitude of the approximation error $E_t = H_t - F_t$ on the given disk as follows:
Using Taylor series expansion for $H_t(z)$, with the coefficients
$a_n(z) = ((-1)^n/n!)H_0^{2n}(z)$
It is natural to approximate them (replacing H_0 by F) and truncate the resulting series to N terms, giving
$F_t(z) := \sum_{n<N} b_n(z)t^n$
where $b_n(z) = ((-1)^n/n!)F^{2n}(z)$
Note that $F_0(z) = F(z)$ and the coefficients b_n can be computed recursively (for $n = 0, ..., N - 1$) using the same recursive relation of a_n above.
The bound $M(E_t)$ follows from Cauchy estimates for the coefficients $a_n(z)$ and $b_n(z)$ (as derivatives of $H_0(z), F(z)$), giving
$M(E_t) \leq \sum_{n<N}[(t^n/n!)(2n)!M(E)/r^{2n}]$
$+ \sum_{n\geq N}[(t^n/n!)(2n)!M(H_0)/r^{2n}]$
Where $M(H_0)$ is an upper bound on $|H_0|$ in the given disk which is upper bounded by $M(F) + M(E)$.
Remark: It seems that for each given z, t, one may take the disk center $z_0 = z$, but the disk radius r and the number of terms N should be optimized (to minimize $M(E_t)$).

Anonymous · 3 February 2018

Correction: in the expressions above for the coefficients $a_n(z), b_n(z)$, the repeated derivatives $H_0^{2n}(z), F^{2n}(z)$ should be $H_0^{(2n)}(z), F^{(2n)}(z)$

Anonymous · 3 February 2018

I just realized that unfortunately, Cauchy derivatives estimates used for the bound are not sufficient (the second sum diverges). The approximant F_t may (hopefully) still be useful, but it seems that a better local estimates for the of magnitude of the derivatives of H_0 are required for a rigorous bound.



meditationatae · 3 February 2018

Michael Rubinstein has a gzipped file with the first 35,161,820 zeros of the Riemann zeta function, as described in the Readme at: http://oto.math.uwaterloo.ca/~mrubinst/L_function_public/ZEROS/Readme.txt . The zipped file zeros_0001_35161820.gz is in the directory: http://oto.math.uwaterloo.ca/~mrubinst/L_function_public/ZEROS/DEGREE_1/ .

He's also the author of "l-calc", an L-function calculator, described in "Software" at http://oto.math.uwaterloo.ca/~mrubinst/L_function_public/L.html

In the past, I've succeeded in building lcalc from source code using "make" in Linux, but today the "build" process encountered a bug/error.



David Bernier (@doubledeckerpot) · 3 February 2018

I changed one line in the Makefile, so as to use the System/Distribution C++ compiler originating from Red Hat. With that modification, the build process using the "make" command completed without a hitch, and as administrator/root, I installed lcalc.

lcalc is very fast: for roughly 12 significant figures, it computes one million Riemann zeta zeros in just over 5 minutes. I'm now running lcalc to compute the first 10 million zeros.

The command and output at the prompt for 1 million zeros were:

$ time lcalc –zeros=1000000 > /dev/null

real 5m8.578s

user 4m55.946s

sys 0m0.758s



David Bernier (@doubledeckerpot) · 4 February 2018

The command:

$ time lcalc -zeros=10000000 > /dev/null ,

which computes the first 10,000,000 zeros of zeta, then redirects the output to "trash" instead of displaying it on the terminal, took about 2 hours to finish.



Sujit Nair · 3 February 2018

Terry, the right side of $F_t(s)$ doesn't make sense if $s = x + iy$.
It took some fiddling around (and going over the asymptotics page) for me to realize that $s = \frac{1+ix-y}{2}$.
Maybe you should define s before the expression for $F_t(s)$.



Terence Tao · 3 February 2018

Actually to evaluate $H_t(x+iy)$ one has to evaluate F_t at two values of s, $s = \frac{1+ix-y}{2}$ and $s = \frac{1+ix+y}{2}$ (see the equation before the definition of $F_t(s)$). So I would prefer to think of s as an independent variable rather than as a quantity defined in terms of x and y.

Sujit Nair · 3 February 2018

If that is the case, shouldn't the summation limit in $F_t(s)$ be in terms of s. Right now, it is $\sqrt{x/4\pi}$?
I am trying to recover Riemann-Siegel for $H_0(x)$ and it is stumping me.

Sujit Nair · 3 February 2018

Ah. I think I got it.
So you mean $\sqrt{\Re(s)/4\pi}$ in the summation limit on the right side?

Sujit Nair · 3 February 2018

Actually, this won't work. For example, computing $H_0(x)$ would require computing $F_0(\frac{1+ix}{2})$. In this case, $\sqrt{\Re(s)/4\pi} = \sqrt{1/8\pi} < 1$.



David Bernier (@doubledeckerpot) · 4 February 2018

In my PARI/gp code for $F_t(s)$, I put $Im(s)/2\pi$ where your expression had $x/4\pi$: https://terrytao.wordpress.com/2018/01/27/polymath15-first-thread-computing-h_t-asymptotics-and-dynamics-of-zeroes/#comment-491988 .
Hopefully, this substitution was correct for the purpose of computing H_t .



Terence Tao · 4 February 2018

Yes, one should have $\mathrm{Im}(s)/2\pi$ instead of $x/4\pi$ there. I'll edit the text accordingly.



KM · 4 February 2018

On running the approx functional eqn (Ht_AFE for brevity) for t=0 between heights T=50000 to 100000 (H_0 has 34517 actual zeros in this range), we get these results
T range————detected—not——actual—-% not detected
50000-60000——6613——58——6671——-0.87%
60000-70000——6773——30——6803——-0.44%
70000-80000——6869——50——6919——-0.72%
80000-90000——6969——48——7017——-0.68%
90000-100000—-7053——-54—-7107———0.76%
Total————34277——240—34517——-0.70%
Whenever zeros were not detected they came in pairs, for eg.
Ht_AFE root–Actual root
54677.865—-54677.8535
54680.645—-54680.66596
————————54681.9331
————————54682.27354
54684.085—-54684.04412
54685.165—-54685.18586
Instead near the undetected ones, Ht_AFE came close to the origin and flipped back forming a local minima or maxima, which I think is due to these zeros being close by. In a plot of actual H_0, its likely the function just crosses the origin before going back.
The computations are being done using a multiprecision library mpmath using 30 significant digits, hence I checked if increasing the precision in this region to 100 digits would help. Using a bisection solver
Solving Ht_AFE between 54679.0 and 54681 -> 54680.642507114... as expected
Solving Ht_AFE between 54681.0 and 54683 -> 54681.0000000000...

Solving Ht_AFE between 54680.65 and 54684.08 -> 54680.650000....
the last two instances showing the solver sticks to the endpoint of the given interval. Similar behavior was observed at another such pair.
I will now run the same exercise at T>2000000, and check the behavior there. As indicated by the AFE formula, Its likely that beyond a certain height such pairs will be either rare or non-existent, as Ht_AFE simulates H_t with higher precision.

Sujit Nair · 4 February 2018

KM, mpmath is a good find!
I assume you are still using the root solver to detect, right?

Anonymous · 5 February 2018

From a numerical viewpoint, If there is a cluster of closely packed zeros of a function $f(z)$, a fast local root solver based on Newton-Raphson algorithm ($z_{k+1} = z_k - f(z_k)/f'(z_k)$) is expected to show such erratic behavior (because the increment $-f(z_k)/f'(z_k)$ can be very large if z_k is very close to a local maximizer of $|f(z)|$ on the real axis – inside the zeros cluster).
A possible solution is to limit the increment magnitude $|z_{k+1} - z_k|$ to be at most 1(say), and then to verify that $|f(z_{k+1})| \leq |f(z_k)|$ (e.g. by dividing the increment by a suitable power of 2). The resulting monotonicity of $|f(z_k)|$ ensures convergence of z_k to a local minimizer of $|f(z)|$ (on the real axis). If complex increments are allowed, then (obviously) if $f(z)$ is analytic, any local minimizer of $|f(z)|$ must be a zero of f.

Anonymous · 5 February 2018

I think using root solving approaches with closely packed zeroes is feasible if done carefully, as you laid out. What also should be considered while doing this is that the zero density gradually increases with T, and the gaps between zeroes can sometimes be much smaller than average gaps in that region.

KM · 5 February 2018

No, this is based on evaluating the function for each z with step size of 0.01, and then counting the sign changes..the intervals have to become finer for larger heights (I am using 0.002 for the T>2 mn exercise). Currently the zero location is taken as the midpoint of an interval where a sign change took place.
The bisection solver was only used to investigate further where a zero pair had been missed. I think such solvers in general can also be used to find the exact roots in the intervals with detected sign changes (thus improving upon the midpoint estimate). The counting method and the solver approach are then complementary.

Sujit Nair · 5 February 2018

Got it. Is there a branch for this on github? I didn't see any after a "git fetch".

Anonymous · 5 February 2018

Well these were some adhoc loops, repeatedly calling the AFE function and recording the output on my PC. I will upload whatever I have as a separate branch.

Anonymous · 5 February 2018

Platt's recent paper (Math. comp. 86(2017), 2449-2467)
https://doi.org/10.1090/mcom/3198

Describes an efficient (using FFT) algorithm for the computation of more than 10^{11} first Zeta zeros with (rigorous) absolute precision of 10^{-30}. Is it possible to extend this algorithm for a large-scale, high-precision computation of H_t zeros?



Anonymous · 5 February 2018

Thanks for the link. Well having a FFT version to compute the AFE at many simultaneous points would be cool. The single point evaluations have become quite slow at larger heights.



David Bernier (@doubledeckerpot) · 4 February 2018

I'm trying to set up a numerical integration expression for the second exact formula in the post for $H_0(2Im(\rho_1))$, where ρ_1 is the first non-trivial zero of zeta.
I tried the same expression with realprecision = 150, and with realprecision = 200. I get respectively roughly 4 e-29 and 1 e-38.
This puzzles me, and I'm wondering if it could be that the integrand takes on large values, but I haven't looked into that.
For reference, the PARI/gp code, along with the output for realprecision = 200 digits are copied below.

```
z=2*z1;
 t=0.0;
 thet=Pi/8 − 1/(4*z);
 K=0.0;
 for(Y=1, 80, b=z-9*I;
 beta=Pi*Y^2;
 K=K+2*Pi*Pi*(Y^4)*sum(J=0,64, intnumgauss(X=J/16, (J+1)/16, exp( t*(X+I*thet)^2 −
beta*exp(4*(X+I*thet)) + I*b*(X+I*thet)), ) ) ;
 b = z-5*I;
 K=K − 3*Pi*(Y^2)*sum(J=0,64, intnumgauss(X=J/16, (J+1)/16, exp( t*(X+I*thet)^2 −
beta*exp(4*(X+I*thet)) + I*b*(X+I*thet)), ) ) );
 print( real((K+conj(K)))/2)
```

Output:
1.08 E-38 // realprecision = 200 digits



David Bernier (@doubledeckerpot) · 4 February 2018

I used the first formula for $K_{t,\theta}(z)$ after equation (2) [1st thread] to evaluate numerically $H_0(2Im(\rho_1))$.
Among the (numerical) expressions in the first thread springing from (2) to (3), it's the one with value closest to zero.

```
Phi(X) = sum(Y=1, 50, (2*Pi*Pi*(Y^4)*exp(9*X) -3*Pi*(Y^2)*exp(5*X))*exp(-Pi*(Y^2)*exp(4*X)))
intnumgaussinit(800); // 800-point Gauss-Legendre method
```

realprecision = 154 significant digits
For the PARI/gp numerical integration code:

```
z=2*z1;
 t=0.0;
 thet=Pi/8 − 1/(4*z);
 K=0.0;
 for(J=0, 256 , K=K+intnumgauss(X=J/128, (J+1)/128, Phi(X+I*thet)*exp(I*z*(X+I*thet)) ));
 print( real((K+conj(K)))/2)
```

output:
1.809 E-127

 **David Bernier (@doubledeckerpot)** · 5 February 2018

It seems at first glance as though the real/imaginary parts of the integrand $H_{0,\theta}$ might decay more rapidly when $\theta = 0$ than when $\theta = \pi/8 - \epsilon$ ...

Below, there is some data on the real part of the integrand for the choice $\theta = 0$:

```
z=2*z1;
 t=0.0;
 thet=0.0;
 for(J=0, 16 ,X=J/16.0;
 K=real(Phi(X+I*thet)*exp(I*z*(X+I*thet)) );
 print(X," ", K))
0.0000000000000000000 0.4466969004671234441
 0.062500000000000000000 -0.07506099482654958609
 0.12500000000000000000 -0.2247326399152970553
 0.18750000000000000000 0.05928707622660076264
 0.25000000000000000000 0.02137269899984563796
 0.31250000000000000000 -0.004075283488938568051
 0.37500000000000000000 -0.0001521228758848166709
 0.43750000000000000000 1.281421125067103168 E-5
 0.50000000000000000000 3.364341936999641644 E-10
 0.56250000000000000000 -3.299802452045272008 E-10
 0.62500000000000000000 4.775197762416083978 E-14
 0.68750000000000000000 3.527315493478114852 E-18
 0.75000000000000000000 -4.571153262716285137 E-24
 0.81250000000000000000 -1.053342402347743622 E-31
 0.87500000000000000000 3.104515713884114793 E-41
 0.93750000000000000000 1.738363069214473369 E-54
 1.00000000000000000000 -5.101940498958935872 E-70
? z
 %101 = 28.26945028346938758
```

 **Sujit Nair** · 5 February 2018

KM, David,

Here is some feedback from Fredrik Johansson (main author of mpmath) to compute H_t with arbitrary precision.

———————-

I was going to recommend implementing the function H_t using Arb (http://arblib.org/). This will be a bit more complicated to code for than mpmath, but it should be much faster, and it will allow obtaining rigorous error bounds.

I recently added code for rigorous numerical integration which is described here

This is using a rigorous, adaptive version of Gauss-Legendre quadrature which should be quite efficient for the type of integrand defining H_t, although there could be some subtleties in practice.

 **Anonymous** · 5 February 2018

Thanks, will try it out.

 **David Bernier (@doubledeckerpot)** · 5 February 2018

Thanks, this is encouraging news.

David Bernier (@doubledeckerpot) · 5 February 2018

An adaptive version of Gauss-Legendre quadrature seems particularly appropriate, at least for H_0, since the integrand decreases extremely rapidly on e.g. the interval [0,2],
cf.:
https://terrytao.wordpress.com/2018/02/02/polymath15-second-thread-generalising-the-riemann-siegel-approximate-functional-equation/#comment-492141 .

David Bernier (@doubledeckerpot) · 5 February 2018

There are two references regarding the Riemann-Siegel formula that I'd like to mention as possibly relevant: the first is Chapter 7 of Edwards' book, and the second is Chapter 3 of Glen Pugh's Master's Thesis at the University of British Columbia, available from the link https://web.viu.ca/pughg/thesis.d/masters.thesis.pdf .

Terence Tao · 5 February 2018

Thanks for this! I am in the process of extending the Riemann-Siegel formula to H_t and trying to compute the next term in the asymptotic expansion. I now see that the main term should only have an accuracy of $O(x^{-1/4})$ (I had previously thought it was $O(x^{-1/2})$) and so a first correction term will probably be needed (which would improve the accuracy to a much more tolerable $O(x^{-3/4})$). It is unfortunately a bit of a mess though and I have already caught a number of sign errors etc. in my calculations, but Pugh's thesis will be a good cross-check. (I have some partial computations on the wiki at http://michaelnielsen.org/polymath1/index.php?title=Asymptotics_of_H_t right now, but it is not yet completed.)

Gil Kalai · 5 February 2018

A side qiestion. Looking at the (impressive) asymptotic expansions and error estimations I wonder if this part could also be automated (a la Zeilberger-Wilf etc.). and not just the numerical work.

K · 6 February 2018

This sounds very interesting, could you please elaborate a bit more on this?

Terence Tao · 5 February 2018

OK, my preliminary computations suggest that a refined asymptotic expansion for $H_t(x+iy)$ is
$H_t(x+iy) \approx A + B + C$
[EDIT: This should be $A + B - C$] where
$A := \frac{1}{8}\frac{s(s-1)}{2}\pi^{-s/2}\Gamma(\frac{s}{2})\sum_{n=1}^{N}\frac{\exp(\frac{t}{16}\log^2\frac{s+4}{2\pi n^2})}{n^s}$
$B := \frac{1}{8}\frac{s(s-1)}{2}\pi^{-(1-s)/2}\Gamma(\frac{1-s}{2})\sum_{m=1}^{M}\frac{\exp(\frac{t}{16}\log^2\frac{5-s}{2\pi m^2})}{m^{1-s}}$
$C := \frac{1}{8}\frac{s(s-1)}{2}\pi^{-s/2}\Gamma(\frac{s}{2})\frac{\Gamma(1-s)e^{-i\pi s}}{2\pi i}(2\pi i M)^{s-1} \times$
$\times \exp(-\frac{t\pi^2}{64})\Psi(\frac{s}{2\pi i M} - N)$
where $s := \frac{1+ix-y}{2}$, N, M are integers with $N, M = \sqrt{\frac{x}{4\pi}} + O(1)$ (for instance one can set $N = M = \lfloor\sqrt{x/4\pi}\rfloor$ if one wishes), one uses the standard branch of the logarithm throughout, and Ψ is the function
$\Psi(\alpha) := 2\pi\frac{\cos(\pi(\alpha^2/2-\alpha-1/8))}{\cos(\pi\alpha)}\exp(\frac{i\pi}{2}\alpha^2 - \frac{5\pi i}{8})$.
This refines the previous approximation, which was essentially $H_t(x+iy) \approx A + B$. Roughly speaking, the first term A should be of size $e^{-\pi x/8}x^{(7-y)/4}\exp(t\log^2 x)$, the

second term B should be of size $e^{-\pi x/8} x^{(7+y)/4} \exp(t \log^2 x)$ (and become dominant once y is significantly larger than $1/\log x$), and the final term C should be of size $e^{-\pi x/8} x^{6/4}$, with the error being of the order of $e^{-\pi x/8} x^{4/4}$ (plus perhaps another error of size $e^{-\pi x/8} x^{3/4} \exp(t \log^2 x)$ that becomes relevant when t is much larger than $1/\log x$).
The function Ψ looks strange, but can also be defined as the unique solution to the functional equations
$\Psi(\alpha) = 2\pi i - e^{-2\pi i \alpha} \Psi(\alpha+1)$
$\Psi(\alpha+1) - \Psi(\alpha) = \frac{2\pi}{\sqrt{i}} \exp(i\pi\alpha^2)$
and is also given by a contour integral
$\Psi(\alpha) = \int_C \frac{\exp(\frac{iz^2}{4\pi} + \alpha z)}{e^z - 1}\, dz$
where C is for instance the contour $t \mapsto \pi i - (1-i)t$ for $-\infty < t < \infty$.
The formula is messy, but can be simplified a little (up to lower order terms) by using the Stirling approximation and collecting some terms. An interesting feature is that one can modify N and M by $O(1)$ without making significant changes to the final approximation, although some of the individual terms in that approximation would change. (This is one way to check that all the signs etc. in the above approximation are accurate.) If one sets $t = 0$ and $y = 0$ then one essentially recovers the classical Riemann-Siegel asymptotics (after extracting the main term of the remainder and discarding the rest).
I will try to write up a rigorous proof of this approximation with an error term (using $O()$ notation with unspecified constants for now, since putting in explicit constants, while ultimately necessary, will make the derivation far more messier). (I started the computations on the wiki but they are unfinished and not properly organised, I will probably end up rewriting it.)

Anonymous · 5 February 2018

Another function Φ is already used in the definition of H_t.
[Good point; I've renamed it Ψ for now. -T.]

Anonymous · 6 February 2018

In the sub-page on H_t asymptotics, the letter t denotes also the imaginary part of s.
[Changed to T – T.]

David Bernier (@doubledeckerpot) · 5 February 2018

I started writing the various formulae , mainly as functions of s, in PARI/gp.
It finds a sign change in $Re(H_0)$ near $x = 30$, $y = 0$, which looks questionable.
I figure I'll take another look at these formulae later.
Function definitions:

```
ga =
 (S)->S*(S-1)*exp(-S*log(Pi)/2)*gamma(S/2)/16
gb =
 (S)->S*(S-1)*exp((S-1)*log(Pi)/2)*gamma((1-S)/2)/16
M =
 (S)->floor(sqrt(imag(S)/(2*Pi)))
A =
 (S)->ga(S)*sum(Y=1,M(S),exp((t/16)*(log((S+4)/(2*Pi*(Y^2)))^2))*exp(-S*log(Y)))
B =
 (S)->gb(S)*sum(Y=1,M(S),exp((t/16)*(log((5-S)/(2*Pi*Y^2))^2))*exp((S-1)*log(Y)))
Phi =
 (Z)->2*Pi*(cos(Pi*((Z^2)/2-Z-1/8))/cos(Pi*Z))*exp(I*(Pi/2)*(Z^2)-(5/8)*Pi*I)
```

C =
(S)->ga(S)*gamma(1-S)*(exp(-I*Pi*S)/(2*Pi*I))*exp((S-1)*log(2*Pi*I*M(S)))*exp(-t*(Pi^2)/64)*Phi(S/(2*Pi*M(S)*I)-M(S))
E =
(S)->A(S)+B(S)+C(S)
H =
(Z)->E(I*real(Z)/2+(1-imag(Z))/2)

 **David Bernier (@doubledeckerpot)** · 5 February 2018

Could there be a sign error in $A + B + C$?
(for my functions based on Terry's calculations), i.e.
Changing from $A + B + C$ to $A + B - C$ improves accuracy,
at least some of the time :
E3(S) = A(S)+B(S)-C(S)
H3(Z) = E3(I*real(Z)/2+(1-imag(Z))/2)
Calculations:
? abs(zeta(1/2+I*74919.075161121))
= 1.44654472644 E-9
? abs(H3(2*74919.075161121))/abs(H(2*74919.075161121))
= 0.000834063509455
I also checked that two zeros of H_0 that were missed in [440, 444] via sign changes in the old formula, without remainder, are found when using the H3 above, near twice the zeta zeros: 220.7 and 221.45 .

 **Anonymous** · 6 February 2018

Is there a Riemann-Siegel integral formula for every H_t?http://mathworld.wolfram.com/Riemann-SiegelIntegralFormula.html

 **David Bernier (@doubledeckerpot)** · 8 February 2018

Glen Pugh writes on one of his web pages:
"Gram's Law is the statement that zeros of Z(t) alternate with the Gram points",
references at: https://web.viu.ca/pughg/RiemannZeta/RiemannZetaLong.html and https://en.wikipedia.org/wiki/Riemann%E2%80%93Siegel_theta_function .
For H_t, could there be an analogue of Gram's Law and/or an analogue of the Riemann-Siegel ϑ function ?

 **David Bernier (@doubledeckerpot)** · 12 February 2018

Could there be an analogue for the H_t of the Hardy function $Z(T)$?
The $Z(.)$ function can be used with Euler-Maclaurin summation to get very accurate values of the zeta zeroes:
https://web.viu.ca/pughg/RiemannZeta/RiemannZetaLong.html#ZetaFunction
Also, Edwards Section 6.4 of Chapter 6.

 **Anonymous** · 5 February 2018

Dear Terry,
There are smoothed approximate functional equations that are exact and provide much better error terms (for instance one can select Gaussian kernels e^{-x^2} in the smooth weights). See for instance the method of proof in https://arxiv.org/pdf/math/0111312.pdf (the argument is folklore). Cutting such a smoothed approximate functional equation at N = \sqrt{T} \log T gives an excellent error term at the

price of a slightly longer computation (by a factor of \log T). Is there a particular reason why the same technique wouldn't go through in the case of H_t ?



Terence Tao · 5 February 2018

There are likely formulae like this – the exact formula in the third display of this blog post is something of this sort, actually (particularly if one applies a smooth splitting of the contour from $i\theta - \infty$ to $i\theta + \infty$ rather than a rough one), but I think these formulae will be slower to compute, even if they are slightly more compact in form. The error term in the Riemann-Siegel formula has an asymptotic expansion whose first few terms decay extremely fast (each term in the expansion is about $T^{-1/2}$ the size of the previous one), and are far easier to compute than the additional $\sqrt{T} \log T - \sqrt{T}$ terms one would need in the smoothed formula.

One way to think about it is this: Poisson summation suggests that any summation of $\frac{1}{n^s}$ in the range $n \sim N$ can be converted to a summation of $\frac{1}{m^{1-s}}$ in the range $m \sim \frac{T}{2\pi N}$, up to some tractable Gamma factors. Naively, the former sum takes about $O(N)$ terms to compute and the latter takes $O(T/2\pi N)$ to compute. One can split the Poisson summation in various ways, but the most efficient thing we can really do is to sum directly for n up to $\sqrt{T/2\pi}$ and then apply Poisson summation beyond that range, for $O(\sqrt{T})$ computations; the rough cutoff introduces an error term but we fortunately have a good asymptotic expansion for it. A smooth cutoff removes this small error term, but at the cost of placing more of the summation on the inefficient side of the Poisson summation formula, and does not seem to be a win in practice (certainly none of the numerical literature on RH verification use these sorts of formulae). Here by the way is a place where numerics diverges from theory, since when computational complexity (or implied constants) are not an issue it is certainly advantageous to smooth sums whenever possible.



David Bernier (@doubledeckerpot) · 6 February 2018

I decided to count zeta zeros via sign changes in the real part of a function I defined in PARI/gp that I called H3, based on Terry Tao's Riemann-Siegel formula calculations.

It's set to look for zeros of zeta with imaginary part below 16,000.

The output is being written to a file, so that I can use it later.

The command that is now running is copied below, and already up to height 4000.0, the count of zeta zeros is complete.

```
last = 1.0;
 count=0;
 for(J=1000,1600000,
 new = real(H3(J/50.0));
 if( (new*last)<0,
 count=count+1;
 write(zeros_to_16k, count," ", J/100.0));
 last = new);
```



David Bernier (@doubledeckerpot) · 6 February 2018

The agreement with tables of zeta zeros using H3, with $A + B - C$ instead of $A + B + C$, is excellent. It seems likely either I made a mistake in defining $C(s)$, or there was a mistake (sign error) in the complex equations...

H3 locates 17423 zeta zeros up to height 16000.0, and tables of zeta zeros say there are in fact 17425. It appears H3 didn't locate zeros 5229.198557199 and 5229.241811259.

Through re-scaling, this appears as two "very near misses" with respect to sign changes:

```
? for(J=1045820, 1045860, print(J/200.0," ", real(H3(J/100.0))*(10^1778)))
```

[...]
5229.18000000 0.0191439033097
5229.18500000 0.0151796159916
5229.19000000 0.0117431220497
5229.19500000 0.00883338972137
5229.20000000 0.00644920191919
5229.20500000 0.00458915818597
5229.21000000 0.00325167668798
5229.21500000 0.00243499624578
5229.22000000 0.00213717840158
5229.22500000 0.00235610952246
5229.23000000 0.00308950293842
5229.23500000 0.00433490111440
5229.24000000 0.00608967785543
5229.24500000 0.00835104054402
5229.25000000 0.0111160324088
[...]



David Bernier (@doubledeckerpot) · 6 February 2018

The 17,425th zeta zero, as determined by a sign change in the real part of H3, is off by only 0.000074 .



Terence Tao · 6 February 2018

It is entirely plausible that I made a sign error. I will not have time to do this until much later today, but I suspect that I managed to forget about the minus sign in the formula for the remainder R(s) in the Riemann-Siegel formula https://en.wikipedia.org/wiki/Riemann%E2%80%93Siegel_formula .



David Bernier (@doubledeckerpot) · 6 February 2018

Ok, thanks! I'm computing another dataset of zeta zeros up to height 64000 (using H3), and writing the output to a file. I'm not sure what the best tests might be for "quality control", i.e. that the data is what it should be theoretically.



KM · 6 February 2018

I too have started a new search for H_0 zeroes with the new eqn (between heights 50k to 100k), and the initial results show a dramatic improvement.
If by quality control you mean number of zeroes, there are 38379 H_0 zeroes from 0 to 64000, so it can be compared against that. Also with the zeroes estimated to two decimal places, the average gap between the estimated and actual zeroes near the 50k mark is close to 0.005.



David Bernier (@doubledeckerpot) · 6 February 2018

The program found, via sign changes in the real part of "H3", 83816 of the 83818 non-trivial zeta zeros rho with imaginary part between 0 and 64,000 .
I plan to use something close to the secant method , https://en.wikipedia.org/wiki/Secant_method , to locate with higher precision the sign changes in the real part of "H3" ...

 **David Bernier (@doubledeckerpot)** · 6 February 2018

The approximate zeta zero, using H_0 with remainder term, at 63999.4809 (via sign change) is off by 0.0004 from the actual zeta zero at 63999.4813 .

 **KM** · 6 February 2018

So the new eqn has detected all sign changes and hence all 15142 zeroes between T=50k and 70k (in terms of zeta zeroes, T between 25k to 35k), hence I have stopped the benchmarking exercise on my side.
Now running computations for t=0.5 and T>2 million. As predicted by theory, we should start seeing both a decrease in average zero spacing and standard deviation of zero spacings, as the zeroes drift downward and become more regularly spaced.
At this point, we can start dividing t ranges between ourselves. For eg. I am planning to explore and generate statistics for t between 0.48 and 0.5 for a start.

 **David Bernier (@doubledeckerpot)** · 7 February 2018

For zeta zeroes with heights between 32k and 64k, I looked at the errors in the approximations using H_0 with the remainder term, as compared to tables of zeta zeroes. The approximate zeta zeroes are based on where the real part of H_0 with remainder, changes sign. My program finds that the average error is 0.000113 .

 **David Bernier (@doubledeckerpot)** · 7 February 2018

Sujit, KM,
I'll send Sujit my Github handle when I find it...
I'm not a Python programmer, but I can try to run your programs, to do a share of the H_t zeros computations.

 **KM** · 8 February 2018

By the way, checking some statistics for t=0.5 and upto first 30000 zeroes
T N_t(T) # detected zeroes of H_t
10000 4519.665 4521
20000 10142.321 10143
30000 16181.355 16182
pretty good match so far.
Also I just realized the (t/16)log(T/4PI) factor, and hence the drift is quite small. So even at T=10^15, we can expect just 1 more zero at t=0.5 compared to at t=0.
Also, as you had noticed, at t=0 there were 2 zeroes undetected with the ABC eqn at 2*5229.198 and 2*5229.242, but it seems at t=0.5, ABC has detected the corresponding zeroes for those ones too.
n H_0.5 zero zeta zero H_0 zero
4765 10457.4445 5229.1985 10458.3971
4766 10458.9976 5229.2418 10458.4836

 **David Bernier (@doubledeckerpot)** · 8 February 2018

https://kommonmann.wordpress.com/ :
That's interesting what you have about $H_{0.4}$. At some point, we might try doing an exact count of zeroes of $H_{0.4}$ in a rectangle, something like a numerical contour integral, as far as I know, similar to what was done before "Turing's trick" was found.
It could be something to think about.



Terence Tao · 6 February 2018

I have located the sign error (I had reversed the orientation of a contour). The correct approximation is indeed $H_t = A + B - C$. Thanks for pointing it out! I am working on replacing the wiki page with corrected (and more logically structured) asymptotics.
[Edit: wiki page now replaced. -T]



meditationatae · 8 February 2018

I put together in a script file h3zeros16k.gp the PARI/gp commands and function definitions for the zeta zeros with height below 16000, based on the code above. Unfortunately, the script won't run: it prints nothing.

But the examples scripts from PARI/gp Headquarters do work, for example lucas.gp to perform Lucas-Lehmer tests.

GP script files can be compiled with the gp2c compiler package. This worked for the example GP script lucas.gp .



PGniewek · 6 February 2018

Dear Terry, a minor nitpick: there appears to be an error in the second equation below the box containing Proposition 1 in this post: the argument of the rightmost exponential function is $\exp(-\pi n^2 u)$ and I think it should be $\exp(-\pi n^2 e^{4u})$. Cheers.
[Corrected, thanks – T.]



Terence Tao · 6 February 2018

Here is a very crude computation that nevertheless indicates that it should be possible to obtain a reasonable value of T for which one can hope to establish the zero-free property $H_t(x+iy)$ whenever $y \geq \varepsilon$ and $x \geq T$ for usable choices of t, ε.

I'll pick $t = 0.4$ and $\varepsilon = 0.4$; this will lead to a bound $\Lambda \leq t + \frac{1}{2}\varepsilon^2 = 0.48$ which is better than 0.5. I'll pretend that the A+B approximation is exact, thus

$$H_t(x+iy) = \frac{1}{8}\frac{s(s-1)}{2}\pi^{-s/2}\Gamma(s/2)\sum_{n\leq N}\frac{\exp(\frac{t}{16}\log^2\frac{s+4}{2\pi n^2})}{n^s}$$
$$+\frac{1}{8}\frac{s(s-1)}{2}\pi^{-(1-s)/2}\Gamma((1-s)/2)\sum_{n\leq N}\frac{\exp(\frac{t}{16}\log^2\frac{5-s}{2\pi n^2})}{n^{1-s}}$$

with $s = \frac{1-y+ix}{2}$. To make $H_t(x+iy)$ non-zero, it suffices to make the first term in the second sum larger in magnitude than the sum of all the other magnitudes put together. Cancelling some factors, and using the approximation $|\Gamma(s/2)| \approx T^{-y/2}|\Gamma((1-s)/2)|$, this amounts to asking that

$$\exp(\tfrac{t}{16}\mathrm{Re}\log^2(5-s))$$
$$< (T/\pi)^{-y/2}\sum_{n\leq N}\frac{\exp(\frac{t}{16}\mathrm{Re}\log^2\frac{s+4}{2\pi n^2})}{n^{(1-y)/2}}$$
$$+\sum_{2\leq n\leq N}\frac{\exp(\frac{t}{16}\mathrm{Re}\log^2\frac{5-y}{2\pi n^2})}{n^{(1+y)/2}}.$$

We approximate $\mathrm{Re}\log^2\frac{s+4}{2\pi n^2}$ as $\log^2\frac{x}{4\pi} - \frac{\pi^2}{4} - 4\log\frac{x}{4\pi n}\log n$ and similarly with $s+4$ replaced by $5-x$, so we reduce to showing that

$$1$$
$$< (T/\pi)^{-y/2}\sum_{n\leq N}\frac{1}{n^{(1-y)/2+\frac{t}{4}\log\frac{x}{4\pi n}}}$$
$$+\sum_{2\leq n\leq N}\frac{1}{n^{(1+y)/2+\frac{t}{4}\log\frac{x}{4\pi n}}}.$$

In order to do a cheap computation, I crudely bounded $\log\frac{x}{4\pi n} \geq \frac{1}{2}\log\frac{x}{4\pi}$, and then extended the sum to all n, giving the criterion

$$1 < (T/\pi)^{y/2}\zeta(\tfrac{1-y}{2} + \tfrac{t}{8}\log\tfrac{T}{4\pi}) + \zeta(\tfrac{1+y}{2} + \tfrac{t}{8}\log\tfrac{T}{4\pi}) - 1.$$

Setting $y = t = 0.4$, it seems that we can make this criterion for $T = 10^{11}$ or so, which is only barely within range of what is numerically verified for RH. One can probably do better if one doesn't bound things quite as crudely. It's also possible that when $t = 0.4$, the sum for H_t is quite rapidly convergent, and one could numerically verify the absence of zeroes for $y \geq 0.4$ for quite a large range

of T, e.g. $10^{11} \leq T \leq 10^{13}$, and then use something like the above argument to handle all larger values of T.

Anonymous · 7 February 2018

In proposition 1 [of the wiki page on asymptotics], the definition of w (which appears in both sides) and the exponent in the LHS, is not sufficiently clear.

Terence Tao · 7 February 2018

I am not quite sure what you are referring to here. There is a definition of the complex exponential w^{s-1}, but this is not a definition of w itself, which is a dummy variable of integration.

Anonymous · 7 February 2018

I was confused because the letter s in the exponent of w^{s-1} appears so tiny and unrecognizable (even under magnification) as the letter s.

Anonymous · 7 February 2018

The letter S in the exponent seems to be distorted(!) in that wiki page (but not here).

Terence Tao · 7 February 2018

Set $B_0(s) := \frac{1}{8}\frac{s(s-1)}{2}\pi^{-(1-s)/2}\Gamma((1-s)/2)\exp(\frac{t}{16}\log^2\frac{5-s}{2\pi})$ for the main term of B. A plausible objective would be to show that for $t = 0.4$ and $y = 0.4$ and $s = \frac{1+ix-y}{2}$, the quantity $A(s) + B(s)$ does not vanish for all x larger than some reasonable threshold T. This is equivalent to $\frac{A(s)+B(s)}{B_0(s)}$ not vanishing; if I did not make any computational errors, this function can be written explicitly as

$$\sum_{n\leq N} \frac{1}{n^{1-s+\frac{t}{4}\log\frac{5-s}{2\pi n}}} + \pi^{\frac{1-2s}{2}}\frac{\Gamma(s/2)}{\Gamma((1-s)/2)}\exp(\tfrac{t}{4}(\log^2\tfrac{s+4}{2\pi} - \log^2\tfrac{5-s}{2\pi}))\sum_{n\leq N}\frac{1}{n^{s+\frac{t}{4}\log\frac{s+4}{2\pi n}}}.$$

In the limit $x \to \infty$ this quantity converges to 1; the question is how fast it converges and at what point it stays a significant distance away from zero (so that we would be able to replace $A + B$ by the full H_t and still get a zero free region). This looks like something that could be amenable to numerics, given that the sums here should converge reasonably quickly once x is moderately large. (In particular, comparative plots of $(A + B)/B_0$, $(A + B - C)/B_0$, and H_t/B_0 for smallish values of x could be quite revealing as to the feasibility of this strategy.)
[Edited to restore some missing denominators of 2π. -T]

David Bernier (@doubledeckerpot) · 7 February 2018

I see that the LHS of the inequality, for $n = 1$, has $log^2(5 - s)$, but that the RHS for 2<= n <=N has an extra 2π as in: $log^2(5 - s)/(2\pi n^2$ ...

Terence Tao · 7 February 2018

Oops, good catch! I think I have restored all the missing denominators of 2π now. (Also I had a 4π which should also have been a 2π.)

KM · 8 February 2018

Some results on (A+B-C)/B0 and (A+B)/B0 computations for t=0.4, y=0.4. Two exercises were run,
one with x range with step size 1.0,
and the other a shorter one with single point evaluations with x = powers of 10

val below refers to the modulus |(A+B)/B0|
x range——minval–maxval–avgval–stdevval
upto 100k—0.179—4.491—1.077—0.455
100k-200k—0.488—3.339—1.053—0.361
200k-300k—0.508—3.049—1.047—0.335
300k-400k—0.517—2.989—1.043—0.321
400k-500k—0.535—2.826—1.041—0.310
500k-600k—0.529—2.757—1.039—0.303
600k-700k—0.548—2.728—1.038—0.296
more granular upto x=6000 (min value at x=29)
x range——minval–maxval–avgval–stdevval
upto 1000—0.179—4.074—1.219—0.782
1000-2000—0.352—4.403—1.164—0.712
2000-3000—0.352—4.050—1.145—0.671
3000-4000—0.338—4.174—1.134—0.640
4000-5000—0.386—4.491—1.128—0.615
5000-6000—0.377—4.327—1.120—0.599
x+yi————-(A+B-C)/B0—-(A+B)/B0
10^2 + 0.4i—-0.80-0.35i—-0.25-0.18i
10^3 + 0.4i—-0.50+0.44i—-0.39+0.51i
10^4 + 0.4i—-0.52+0.01i—-0.52-0.01i
10^5 + 0.4i—-1.28-0.15i—-1.28-0.16i
10^6 + 0.4i—-1.29-0.76i—-1.29-0.76i
10^7 + 0.4i—-1.54+0.13i—-1.54+0.13i
10^8 + 0.4i—-1.47-0.09i—-1.47-0.09i
10^9 + 0.4i—-1.23+0.18i—-1.23+0.18i
10^10 + 0.4i—-0.86+0.07i—-0.86+0.07i
10^11 + 0.4i—-1.09-0.07i—-1.09-0.07i
10^12 + 0.4i—-0.996+0.09i—0.996+0.09i
10^13 + 0.4i—-0.953+0.045i–0.953+0.045i
It seems (A+B)/B0 starts staying away from zero after a very short range, but converges to 1 quite slowly.



Terence Tao · 9 February 2018

This is very promising numerical data! It suggests for instance that the C error term is largely irrelevant for $x \geq 10^4$ or so. One could now hope to establish that $H_t(x+iy) \neq 0$ for all x and $y = 0.4$ by doing a numerical verification up to, say, $x = 10^6$, and then for $x > 10^6$, showing that $|A+B|/B_0 \geq 0.4$ (which seems plausible from the numerical data, and can hopefully be proven by some combination of numerics and analysis) and that $|H_t - (A+B)|/B_0 < 0.4$ (e.g. by showing $|H_t - (A+B-C)|/B_0 < 0.3$ and $|C|/B_0 < 0.1$). If $(A+B)/B_0$ not only stays away from the origin, but also from the negative real axis, then the argument principle (and Rouche's theorem) should also preclude zeroes in the region $y > 0.4$ as well (one may also have to control what goes on at some large value of y, e.g. $y = 10$, but I think this should be very easy).



Anonymous · 9 February 2018

Since $H_t(x+iy) \neq 0$ for any $t \geq 0$ and $y \geq 1$, it is not clear why larger values of y should be considered?



Terence Tao · 10 February 2018

To apply the argument principle to establish a zero-free region such as $\{x+iy : T_1 \leq |x| \leq T_2, y \geq 0.4\}$, it is not enough to know that the function $H_t(x+iy)$ is non-zero on the boundary of the region; one also must control the winding number. For y very large (I conservatively put $y=10$, but one can probably do much better) one can approximate $H_t(x+iy)$ quite well by B_0 whose contribution to the winding number is very well controlled (coming almost entirely from the gamma function factor). One may still have to control $H_t(x+iy)$ for say $0.4 \leq y \leq 10$ on the $x = T_1, T_2$ sides, but probably we can take T_1 small enough that this verification is easy, and for T_2 large enough perhaps the analysis estimates would suffice.



KM · 10 February 2018

Some results for t=0.4, y=0.4 for Real((A+B)/B0) and |C/B0|. Every point n+yi was evaluated, where n is a natural number (showing results till n = 0.7 million)
While it may be possible for (A+B)/B0 and C/B0 to have different behavior between these points, I think that's unlikely since n+0.4i is effectively random for H_t, and the sample size of observations is large.
It seems Real((A+B)/B0) stays positive beyond a small height and afterwards becomes comfortably above 0.4 and keeps moving towards 1. This reflects also in |(A+B)/B0|
n−−−−−minval−maxval
upto 1000—(0.09)—4.06
1000-2000—0.02—−4.33
2000-3000—0.15—−3.99
3000-4000—0.19—−3.99
4000-5000—0.34—−4.48
5000-6000—0.33—−4.33
6000-7000—0.35—−3.88
−−−−−−−−−−−−
upto 100k—(0.09)—4.48
100k-200k—0.48—−3.32
200k-300k—0.50—−3.00
300k-400k—0.52—−2.97
400k-500k—0.53—−2.82
500k-600k—0.53—−2.75
600k-700k—0.55—−2.72
|C/B0| keeps moving towards 0 and is less than 0.1 beyond a small height.
n−−−−−minval−maxval
upto 1000—0.058—2.125
1000-2000—0.036—0.160
2000-3000—0.028—0.095
3000-4000—0.023—0.071
4000-5000—0.020—0.060
5000-6000—0.018—0.052
6000-7000—0.016—0.045
−−−−−−−−−−−−−−-
upto 100k—0.002236—2.124627
100k-200k—0.001262—0.005360
200k-300k—0.000888—0.003030
300k-400k—0.000699—0.002158

400k-500k—0.000572—0.001681
500k-600k—0.000485—0.001382
600k-700k—0.000424—0.001176

As a contrast, I also ran the same exercise for t=0.05 and y=0.4. It seems for t=0.05 to show similar behavior as t=0.4, we will have to go to much larger heights as expected.

|(A+B)/B0|
n————–minval—maxval
upto 100k—0.092—9.231
100k-200k—0.172—10.001
200k-300k—0.160—10.461
300k-400k—0.185—10.177
400k-500k—0.158—9.925
500k-600k—0.183—9.826
600k-700k—0.188—9.928

Real((A+B)/B0)
n————–minval—maxval
upto 100k—(1.24)—8.89
100k-200k—(1.29)—9.98
200k-300k—(1.35)—10.34
300k-400k—(1.17)—10.16
400k-500k—(1.22)—9.91
500k-600k—(0.94)—9.64
600k-700k—(1.23)—9.85

|C/B0|
upto 100k—0.0130—2.2336
100k-200k—0.0097—0.0315
200k-300k—0.0082—0.0236
300k-400k—0.0073—0.0200
400k-500k—0.0067—0.0177
500k-600k—0.0061—0.0161
600k-700k—0.0058—0.0149

Computing H_t independently of the ABC approach at such large heights (for eg. using the integral) is quite tricky and expensive, as shown by David in some of the comments below. We may need a Euler Maclaurin type formula, which was being attempted in one of the threads some time back.

 **KM** · 11 February 2018

Evaluating H_t(x+iy) for t=0.4 and y=0.8,1.2,...4.4 upto height 50k (for integer x) gives these results,

min value of Real((A+B)/B0)
——x range
y—-upto10k-10k20k-20k30k-30k40k-40k50k
————————————————————
0.8—-0.19——0.52——0.55——0.55——0.57
1.2—-0.38——0.60——0.61——0.62——0.64
1.6—-0.47——0.65——0.67——0.67——0.69
2.0—-0.55——0.70——0.72——0.72——0.73
2.4—-0.61——0.74——0.76——0.76——0.77
2.8—-0.67——0.78——0.79——0.80——0.80

3.2—-0.71—-0.81—-0.82—-0.82—-0.83
3.6—-0.74—-0.84—-0.85—-0.85—-0.85
4.0—-0.77—-0.86—-0.87—-0.87—-0.87
4.4—-0.80—-0.88—-0.89—-0.89—-0.89
min value of |H/B0|
——x range
y—-upto10k-10k20k-20k30k-30k40k-40k50k
————————————————————
0.8—-0.18—-0.52—-0.55—-0.56—-0.58
1.2—-0.15—-0.60—-0.62—-0.62—-0.64
1.6—-0.20—-0.66—-0.67—-0.68—-0.69
2.0—-0.35—-0.70—-0.72—-0.72—-0.73
2.4—-0.48—-0.75—-0.76—-0.76—-0.77
2.8—-0.59—-0.78—-0.79—-0.80—-0.80
3.2—-0.67—-0.81—-0.82—-0.83—-0.83
3.6—-0.74—-0.84—-0.85—-0.85—-0.85
4.0—-0.78—-0.86—-0.87—-0.87—-0.87
4.4—-0.82—-0.88—-0.89—-0.89—-0.89
As expected, the behavior seen at t=0.4,y=0.4 becomes more pronounced as y is increased, and the above values converge to 1 faster. Also, Real((A+B)/B0) never went negative in this exercise.

Anonymous · 7 February 2018

In the wiki page on H_t asymptotics (just above "approximation for $t > 0$"), it seems that in the exact expression of $R_{t,N,M}(s)$, the second sum (unlike the first sum) is independent of t.
[Corrected, thanks – T.]

Anonymous · 8 February 2018

Is it possible (by obtaining more information on the dynamics of $\sigma_{max}(t)$) to improve the upper bound for Λ in proposition 1 above (e.g. by reducing the factor $1/2$ in this bound for certain values of t_0 and ϵ)?

KM · 8 February 2018

For t=0.5, we see evidence of the zeros getting quite regularly spaced compared to the the zeros for t=0, even while the average zero gap remains almost the same.
T range—# of———avg—-avg———-stdev—stdev
————-H_0.5—-zero—zero——zero—-zero
————-zeroes—-gap—-gap——gap——gap
——————————H_0.5—-H_0—-H_0.5—-H_0
————————————————————
upto 10k—4521——2.21——2.21——0.65——1.00
10k-20k—5622——1.78——1.78——0.23——0.71
20k-30k—6039——1.66——1.66——0.18——0.66
30k-40k—6310——1.58——1.58——0.16——0.63

Doug Stoll · 9 February 2018

I just found these blogs today. For all of you looking to confirm the Ho zeros here is potentially helpful hint. For Riemann zeta zeros one can use the following for a very "quick and rather good"

approximation to the kth zero. I derived and tested this myself several months ago but have not published it nor have I seen it anywhere.
zz[k] ~ 2e*Pi*A/(ProdLog[A]) +Pi/k – (4.7159-0.3538loglog(k+1))/log(k+1)
Where A = (8k-7)/8e and ProdLog [A] = W[A] whichever notation you are more used to.
It works extremely well even out to 10^24 as I compared it to the published sets of zeros. It is not perfect as it presumes monotonically decreasing spacing but it is close.
So I would expect you to look at 2X this value +/- a little. It would be very informative to see if there is a "consistent" decrement or a "simple function" from 2X towards the origin the further up the i-axis one goes.
And a caution, be careful of your step size exactly for the reason the zeros still will crowd together. You should use a decreasing test step size. To get a feel for the stats of the first 10^13 zeta zeros and particularly the ones that get very close together (and therefore have an impact on the dB-N constant I refer you to:
http://numbers.computation.free.fr/Constants/Miscellaneous/zetazeroscompute.html –
particularly to published samples of zeros explored out to 10^24. Notice that the smallest spacing in your current range of interest (to 10^12) is 0.0001008 at T = 18,403,609,310.0854 (60,917,681,409th zeta zero). The smallest you should find in your test out to 160000 should be the 97,484th zero at 73,235.205353
I have heuristically found an estimate (unpublished) over the range of zz's found to date that the minimum spacing for zeta zero's is larger than:
min gap > 10^{[loglog[T]^3/2 + Pi/2]}
and the maximum spacing is:
max gap < 2/3*loglog[T]^3/2+1)*expected gap where the expected gap can be found from above as zz[k+1]-zz[k].
This is all extremely exciting if you improve Odlyzko's 1999 value for lamda's lower bound of -2.7*10^-9. This is feeling like it could be 0. What an interesting find that would be. Prove RH and show it is just barely true as expected.



Michael Ruxton · 9 February 2018

Didn't this thread start with Rodgers and Tao proving lambda >= 0?



Doug Stoll · 9 February 2018

No – proving it was non-negative – but allowing for 0,



David Bernier (@doubledeckerpot) · 9 February 2018

I'd recommend taking a look at the Proposal post written by Terry Tao here: https://terrytao.wordpress.com/2018/01/24/polymath-proposal-upper-bounding-the-de-bruijn-newman-constant/ .



David Bernier (@doubledeckerpot) · 10 February 2018

Using an 800-point Gauss-Legendre method,
I can accurately evaluate the integral
for H_0(2*z200), where z200 is the 200th zero of Riemann zeta.
By optimizing over the parameters, I got it down to
14 seconds.
The real precision was set to 140 digits, because
there is a lot of cancellation, and the tail
oscillations are extremely tiny.
Below, x6 := 2*z200 :

```
? intnumgaussinit(800); // 40 seconds to initialize 800-point method
 time = 41,428 ms.
? s=0.0;for(J=0,16, s=s+intnumgauss(X=J/12,(J+1)/12, Phi(X)*cos(x6*X),)); s
 time = 13,778 ms.
 %95 = 3.071442230572020945 76[...] E-141 // 14 seconds elapsed time
? x6
 %96 = 792.763708446000000000[...]00000000000000000000
? zeta(1/2+I*x6/2)
 time = 92 ms.
%97 = -7.0548531809481797311234[...] E-10 + 7.029082149944[...] E-10*I
? \p
 realprecision = 154 significant digits (140 digits displayed)
Phi(X):= sum(Y=1,50,(2*Pi*Pi*(Y^4)*exp(9*X)-3*Pi*(Y^2)*exp(5*X))*exp(-Pi*(Y^2)*exp(4*X)))
```



David Bernier (@doubledeckerpot) · 10 February 2018

For z400, I can get the time down to 38 seconds
 to compute accurately H_0(2*z400).
It seems pretty clear that the main factor in the time taken
 is the increasing real precision needed, when
 passing from 200th to 400th and higher.
The table below is a first approximation
 to the time needed to compute accurately H_0(2*z_k) using the integral,
 where z_k is the k'th zero of zeta:
k=1000 time=2.36861 minutes
 k=2000 time=6.42653 minutes
 k=3000 time=11.5225 minutes
 k=4000 time=17.4365 minutes
 k=5000 time=24.0442 minutes
 k=6000 time=31.2631 minutes
 k=7000 time=39.0333 minutes
 k=8000 time=47.3089 minutes
 k=9000 time=56.0535 minutes
 k=10000 time=65.2369 minutes
==========
Commands to PARI/gp and output:

```
? z400=679.742197883;
? \p 38
 realprecision = 38 significant digits
? zeta(1/2+I*z400)
 %153 = -1.269246632[...] E-9 + 4.28823209915[...] E-9*I
? \p 240
 realprecision = 250 significant digits (240 digits displayed)
? z400=679.742197883;
 ? x8=2*z400;
? s=0.0;for(J=0,14, s=s+intnumgauss(X=J/11,(J+1)/11, Phi(X)*cos(x8*X),)); s
 %157 = 7.88880866256[...] E-237
? s=0.0;for(J=0,14, s=s+intnumgauss(X=J/11,(J+1)/11, Phi(X)*cos((x8+1)*X),)); s
 %158 = 5.28032674733[...] E-228
```

? ##
*** last result computed in 37,923 ms.

 **David Bernier (@doubledeckerpot)** · 10 February 2018

For z800, the 800th non-trivial zeta zero, the time to compute
the integral for H_0(2*z800) grows to almost 3 minutes.
Below, the assumption is that time is proportional
to $k^{2.211}$. Then beyond the 10000th zeta
zero, the time to compute $H_0(2z_k)$
using the integral formula becomes prohibitive:
k=1000 time=4.80261 minutes
k=2000 time=22.2359 minutes
k=3000 time=54.4995 minutes
k=4000 time=102.951 minutes
k=5000 time=168.617 minutes
k=6000 time=252.331 minutes
k=7000 time=354.805 minutes
k=8000 time=476.661 minutes
k=9000 time=618.455 minutes
k=10000 time=780.688 minutes
==========
Commands to PARI/gp and output for z800:
? z800=1183.712775296;
? x10=2*z800;
? \p 410
? #
timer = 1 (on)
? s=0.0;for(J=0,15, s=s+intnumgauss(X=J/11,(J+1)/11, Phi(X)*cos((x10)*X),));s
time = 2min, 55,979 ms.
%179 = 7.096[...] E-410

 **Anonymous** · 10 February 2018

In the wiki page on the dynamics of simple zeros, the factor -2 in the RHS of the result should be 2.
By the way, the simplifying assumption (for the Hadamard factorization) that H_t has no zero at the origin, is not needed (since it leads to the same final result.)
[Corrected, thanks – T.]

 **a** · 11 February 2018

An attempt from UChicago on Hilbert's eighth https://arxiv.org/pdf/1708.02653.pdf http://faculty.chicagobooth.edu/nicholas.polson/research/polson-hilbert-8.pdf.

 **Anonymous** · 11 February 2018

In the arxiv paper, the function ψ (also known as Digamma), defined by (21) as the logarithmic derivative of the Gamma function, should have its poles at the poles of Γ (i.e. the nonpositive integers). But by (63), these poles should coincide with the zeros of $\xi(1/2 + s^{1/2})$ which are known to have quasi-random properties. So there must be an error in the derivation of (63).

Anonymous · 6 April 2018

Great catch! But I asked Polson about it and apparently (63) was not intended to be the digamma function — it was just an unfortunate choice of notation. The 7th version (April 1st) on arXiv has ψ_{α} instead of ψ, but still this is apparently defining a new function, and is not claimed to be the digamma function.



David Bernier (@doubledeckerpot) · 11 February 2018

There's an expository paper by Andrew Booker that discusses Turing's method and it's use in the verification of the Riemann Hypothesis to some height. It would be nice if Turing's method could be generalized to the functions H_t . Here's a link to Andrew Booker's paper:
http://www.homepages.ucl.ac.uk/~ucahipe/turing1.pdf .

Anonymous · 11 February 2018

The ODE dynamics of the simple zeros of H_t for $t > 0$ is based on the fact that H_t (in addition of obeying the Bacwards heat equation) is an even entire function of order 1 (allowing simple Hadamard factorization). Is H_t of order 1 also for some $t < 0$ values?



Terence Tao · 11 February 2018

H_t is of order 1 for all t (see the Ki-Kim-Lee paper).



Bob Robbins · 11 February 2018

Wanna work on the Robbinsian method for finding new primes? I found the pattern.



zgtzsj · 12 February 2018

Would you like to share it?



KM · 11 February 2018

For t>0, a simplified form of Gram's type law works quite well at larger T heights for quickly detecting roots. It takes advantage of the fact that zero locations get more predictable as t or T increase.
Using this I was able to compute for both t=0.4 and t=0.5, 85031 zeroes each between T = 500k and 600k. N_t estimate for both is around 85030.67
If after finding the first few zeroes in a region using the classical approach we see that that the stdev(zero gaps)/avg_zero_gap metric is less than a certain value, for eg. 10%, which would imply an interval of avg_gap/2 would cover 5 standard deviations, then such a method can be used. The zero gaps are not seen to follow a normal distribution and have a fatter tail to the right hand side, but the fatter tail shrinks as t or T are increased.
The pseudoalgo works in a 3 step loop,
1) Take a known root z_(t,i-1) and form a search interval
[z_(t,i-1) + avg_expected_gap/2, z_(t,i-1) + 3*avg_expected_gap/2]
2) Evaluate H_t at the interval endpoints and detect if there is a sign change
3) If a sign change exists, locate the root z_(t,i) using an interval solver
Locating z_(t,i) and using that to form the next search interval is necessary. For eg. we can't use [z_(t,i-1) + 3*avg_gap/2, z_(t,i-1) + 5*avg_gap/2] and so on, since if there is a series of smaller or larger gaps, predetermined search intervals fail to detect all the roots. Also avg_expected_gap can be changed at every step, for eg. using [T-0.9T]/[N_t(T) – N_t(0.9T)]



Polymath15, third thread: computing and approximating H_t | What's new · 12 February 2018

[…] thread of the Polymath15 project to upper bound the de Bruijn-Newman constant , continuing this previous thread. Discussion of the project of a non-research nature can continue for now in the existing proposal […]

El proyecto Polymath15 logra reducir la cota superior de la constante de Bruijn-Newman | Ciencia | La Ciencia de la Mula Francis · 26 May 2018

[…] parte real de sus ceros determina una cota superior para el valor de Λ). En el segundo hilo [Tao, 02 Feb 2018] se decidió atacar el problema empezando por una versión sencilla (test problem) con […]